

Embedded Knowledge Base

ESP32 Draggy Racing Logger 工程知识库

Workspace Engineering Notes

2026-06-10

目录

1 Embedded Knowledge Base	4
1.1 目录	4
1.2 阅读前先知道的事实	4
2 第一部分：项目全景图	5
3 ESP32 Draggy Racing Logger	5
3.1 项目背景	5
3.2 项目目标	5
3.3 技术栈	6
3.4 项目一句话总结	6
3.5 项目目录结构分析	6
4 第二部分：系统架构设计	7
5 系统架构	7
5.1 总体架构	7
5.2 为什么这样拆分	7
5.3 数据链路	8
5.3.1 运动数据链	8
5.3.2 展示链	8
5.3.3 存储链	8
6 第三部分：项目运行流程	9
7 上电初始化	9
8 主循环调度	9
9 中断流程	10

10 通信流程	10
10.1 GPS	10
10.2 Web	11
11 状态机	11
11.1 音频录制	11
11.2 性能测试	11
12 第四部分：核心模块解析	12
13 main.cpp：系统装配与输出层	12
14 mpu6050：高振动车辆 IMU 前端	12
15 gps_module：GPS、速度与性能计时	12
16 performance_analyzer：派生指标与摩托语义	13
17 audio_recorder：低码率事件录音	13
18 sd_logger：行驶 CSV	14
19 data/index.html：无 App Web 仪表盘	14
20 cad/pcb1_enclosure.scad：参数化产品外壳	14
21 第五部分：技术知识提取	15
22 双 I2C	15
22.1 为什么需要它	15
22.2 项目中的实际应用	15
22.3 常见 Bug 与调试	15
23 UART 与 NMEA	15
24 采样与数字滤波	15
25 GPS 静止漂移	16
26 GPS-IMU 融合	16
27 ADC、Timer 与音频	16
28 SPI、SD 与文件系统	17
29 Wi-Fi 与 HTTP	17
30 第六部分：设计决策分析	17

目录	3
31 双 I2C 而不是共享总线	17
32 协作式调度而不是 FreeRTOS 任务	17
33 多级滤波而不是单一滤波器	18
34 互补滤波而不是 EKF	18
35 阈值状态机	18
36 浏览器端导出	18
37 第七部分：调试与踩坑记录	19
38 调试记录 1：文档与固件接线冲突	19
39 调试记录 2：音频丢采样	19
40 调试记录 3：SD 功能看似存在但不工作	19
41 调试记录 4：倾角动态响应可能错误	19
42 调试记录 5：GPS 模式不一致	20
43 调试记录 6：启动顺序与 SPIFFS	20
44 调试记录 7：融合字段与 API 不一致	20
45 调试记录 8：HTTP 文件下载路径	20
46 调试记录 9：字符串与缓冲区	20
47 调试记录 10：编码损坏	21
48 第八部分：工程实践总结	21
48.1 做得好的地方	21
48.2 可以改进的地方	21
48.2.1 代码组织	21
48.2.2 配置管理	21
48.2.3 日志	22
48.2.4 错误处理	22
48.2.5 测试策略	22
48.2.6 构建与 CI	22
49 第九部分：项目复盘	23
49.1 最大的亮点	23
49.2 最大的难点	23

49.3 学到的知识	23
49.4 如果重新设计	23
49.5 下一步扩展	23
50 第十部分：个人知识地图	24
51 Embedded Knowledge Map	24
51.1 长期维护入口	26

1 Embedded Knowledge Base

面向未来自己的工程复盘。本文以当前工作区源码为事实基线；旧 README、优化报告和面试笔记只作为设计背景，不覆盖代码事实。

最后核查：2026-06-10

1.1 目录

1. 第一部分：项目全景图
2. 第二部分：系统架构设计
3. 第三部分：项目运行流程
4. 第四部分：核心模块解析
5. 第五部分：技术知识提取
6. 第六部分：设计决策分析
7. 第七部分：调试与踩坑记录
8. 第八部分：工程实践总结
9. 第九部分：项目复盘
10. 第十部分：个人知识地图

1.2 阅读前先知道的事实

这个仓库只有一个主固件项目，但它经历了明显演化：

1. 最初是 ESP32 + MPU6050 + OLED 的双 I2C 演示。
2. 后来加入 GT-U7 GPS，目标变成低成本 Dragy 风格车辆性能仪。
3. 再加入摩托车倾角/G 力分析、Wi-Fi 网页、音频录制、SD 卡日志、PCB 和 3D 打印外壳。

因此旧文档与当前代码有冲突。以后判断“现在是什么”，优先级应为：

当前源码与 platformio.ini

>

当前硬件接线和实测记录

>

README / GPS 优化报告 / 面试笔记

当前已发现的版本漂移：

主题	当前源码	旧文档中的其他说法
I2C	MPU: GPIO21/22; OLED: GPIO18/19, 两条总线	README 曾写 21/19 与 25/26; GPS_
GPS 串口	UART2, GPIO16/17, 9600 bps	优化报告写过 38400 bps
GPS 更新率	启动配置 10 Hz; 掉线恢复时改为 5 Hz	文档同时出现 5 Hz、10 Hz
SD 卡	模块完整, 但 <code>setup()</code> 初始化被注释	部分描述把它写成可直接使用
GPS-IMU 融合	<code>loop()</code> 每 5 ms 调用 <code>fuseIMUSpeed()</code>	/data 的 <code>imuSpeed</code> 仍固定输出 0, 注释称
摩托互补滤波	已在 <code>performance_analyzer.cpp</code> 实现	面试笔记部分段落称姿态互补滤波未实现

编码方面, 大量中文文件显示为 mojibake, 说明文件很可能曾以 UTF-8 保存、后被错误编码链处理。代码符号仍可读, 但这是长期维护风险。

2 第一部分：项目全景图

3 ESP32 Draggy Racing Logger

3.1 项目背景

项目想用低成本器件做一个车载/摩托车数据终端：实时读取车辆运动状态，显示速度、姿态、G 力和性能测试结果，并让手机不安装 App 也能查看数据。

真正困难的不是“读出传感器”，而是车辆环境：

- 发动机和路面引入高频振动。
- MPU6050 有零偏、温漂和安装坐标误差。
- GT-U7 在静止和弱信号时会速度漂移。
- GPS 只有低频绝对速度，IMU 有高频响应但积分漂移。
- Web、OLED、采样、录音和存储都跑在同一个 Arduino 主循环中。

3.2 项目目标

当前代码试图实现：

- 200 Hz IMU 采样与滤波。

- 10 Hz GPS 解析、静止抑制和性能计时。
- GPS-IMU 速度与 G 力的轻量融合。
- 摩托车倾角、弯道、翘头/翘尾和峰值统计。
- OLED 本地显示。
- ESP32 SoftAP + HTTP Web 仪表盘。
- SPIFFS 低码率 WAV 录音。
- SD 卡 10 Hz CSV 行驶日志。
- 与 PCB 配套的参数化 OpenSCAD 外壳。

必须区分：SD 记录代码存在，但当前启动流程没有初始化 SD 卡，所以默认不会真正记录。

3.3 技术栈

类别	使用内容
MCU	ESP32-WROOM-32 / PlatformIO esp32dev
框架	Arduino Framework
语言	C++、HTML/CSS/JavaScript、OpenSCAD
传感器	MPU6050、GT-U7 GPS、模拟麦克风
显示	SSD1306 128x64 OLED
总线	双 I2C、UART2、HSPI、ADC1
网络	Wi-Fi SoftAP、HTTP WebServer
存储	SPIFFS、SD/FAT CSV
数据格式	NMEA、JSON、CSV、PCM WAV
构建	PlatformIO
机械	Gerber、OpenSCAD、STL

3.4 项目一句话总结

基于 ESP32 构建低成本车载多传感器性能记录仪，融合 200 Hz IMU 与 10 Hz GPS，实现姿态/G 力分析、性能计时、Web 实时可视化、录音与可扩展数据记录。

3.5 项目目录结构分析

```
esp32-imu/
├── src/                固件主源码
│   ├── main.cpp       系统装配、调度、OLED、HTTP API
│   ├── mpu6050.*       IMU 驱动、滤波、基础姿态
│   ├── gps_module.*    GPS、性能测试、速度融合、会话持久化
│   ├── performance_analyzer.* G 力融合、车辆行为、摩托车分析
│   ├── audio_recorder.* ADC 定时采样、WAV、自动录音
│   └── sd_logger.*     SD 初始化与 CSV 日志
```

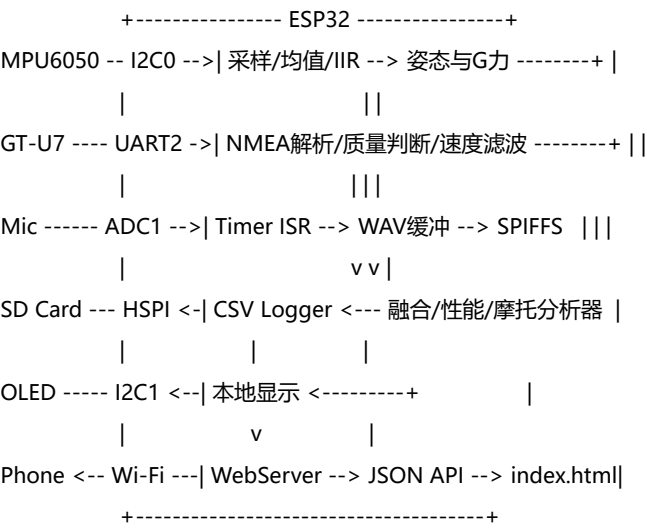
— data/index.html	SPIFFS Web 仪表盘
— cad/	PCB 外壳源码与 STL
— platformio.ini	板卡、串口、依赖、SPIFFS 配置
— README.md	早期项目说明，已落后于源码
— BUILD_GUIDE.md	PlatformIO 构建与排错说明
— GPS_*.md	GPS 接线、升级与优化思路
— GT-U7_*.md	GT-U7 优化总结，部分内容未与代码同步
— INTERVIEW_NOTES.md	技术复盘素材，不是运行事实来源
— Gerber_*.zip	PCB 制造文件
— wiring_diagram.txt	接线说明，接近当前双 I2C 方案
— include/ lib/ test/	PlatformIO 模板目录，暂无自有实现/测试
— interview/	LaTeX/PDF 派生文档，不属于固件运行链

PCB 和外壳是同一产品化尝试的一部分，不应误识别为独立软件项目。Gerber 表明存在双层 PCB、丝印、阻焊、钻孔和飞针测试数据；无法仅从当前工作区确定实际焊接版本和最终 BOM。

4 第二部分：系统架构设计

5 系统架构

5.1 总体架构



5.2 为什么这样拆分

- main.cpp 是 composition root：拥有硬件实例、启动顺序和时间调度。
- 驱动/算法按数据源拆分：IMU、GPS、音频、SD 各自维护状态。
- performance_analyzer 位于传感器之上，读取全局 IMU/GPS 数据形成派生指标。

- OLED 和 HTTP 是两个输出适配器，共享同一批全局状态。

这套拆法适合原型开发，新增功能快；代价是模块通过 `extern` 全局结构体耦合，调用顺序就是隐含的数据一致性协议。

5.3 数据链路

5.3.1 运动数据链

MPU6050 raw acceleration/gyro

|

v

10点移动平均 + 一阶IIR

|

+-> Roll/Pitch

+-> G力/驾驶状态

+-> 摩托倾角互补滤波

+-> IMU速度积分

\

GT-U7 NMEA --> GPS速度 ---> 轻量互补修正 --> 性能指标/Web/OLED/CSV

GPS 提供长期不漂移的绝对速度；IMU 提供短期快速变化。当前实现不是严格卡尔曼滤波，而是偏置校准、死区、积分和 GPS 比例拉回。

5.3.2 展示链

全局状态结构体

+-> OLED: 20 Hz, 五种页面

+-> /data: Web主仪表盘

+-> /moto: 摩托数据

+-> /audio: 录音状态

+-> Serial: 5 Hz简报 / 2 s详细信息

5.3.3 存储链

Mic ADC --> 8-bit/8 kHz PCM --> 512 B buffer --> SPIFFS WAV

GPS+IMU+Fusion --> RideDataPoint --> CSV line --> SD card

Browser samples --> JavaScript memory --> client-side CSV/JSON export

这里实际上有三套“记录”：SPIFFS 音频、SD 原始/融合数据、浏览器内存记录。它们没有统一会话 ID 和时间基准。

6 第三部分：项目运行流程

7 上电初始化

```

Reset
|
v
Serial 115200 + BOOT输入
|
v
双I2C 400 kHz初始化 -> 扫描设备
|
v
MPU6050 -> GPS -> 性能分析 -> 摩托模式 -> 音频
|
v
SD初始化跳过（当前测试模式）
|
v
OLED -> SPIFFS -> Wi-Fi SoftAP -> HTTP路由
|
v
显示连接信息，进入 loop()

```

值得注意：initAudioRecorder() 在 SPIFFS.begin(true) 之前读取 SPIFFS 容量。首次启动或文件系统异常时，这个顺序不稳妥，应该先挂载文件系统。

8 主循环调度

项目没有 FreeRTOS 显式任务，也没有统一 scheduler，而是 millis() 驱动的协作式时间片：

```

loop()
|
+--> server.handleClient()
+--> BOOT按键与串口命令
|
+--> 每5 ms:
|   ReadMPU6050()
|   fuselMUSpeed()
|
+--> 每100 ms:
|   updateGPSData()
|   checkAutoLogging()
|   optional logDataPoint()

```

```

|
+--> 每次循环:
|   updateAudioRecorder()
|
+--> 每50 ms:
    updateFusedData()
    update OLED
    print serial

```

优点是易读、没有任务同步成本。问题是任何阻塞操作都会破坏所有“频率承诺”。HTTP 文件传输、GPS 重连 `delay()`、OLED I2C、SD flush 和串口输出都可能让 5 ms/125 us 级任务失约。

9 中断流程

唯一明确的用户中断路径是音频定时器：

```

125 us timer
|
v
ISR: adc1_get_raw()
|
+--> lastSample = sample
+--> sampleReady = true
    |
    v
主循环 processSample()
|
+--> 12 bit转8 bit
+--> 更新音量
+--> 写入512 B缓冲
+--> 缓冲满后写SPIFFS

```

这不是可靠的生产者-消费者队列。若主循环在两个中断之间没有消费，`lastSample` 被覆盖，只保留最新样本。

10 通信流程

10.1 GPS

```

UART2 bytes
-> TinyGPS++ encode
-> location/speed/time/satellite/HDOP
-> 合理范围与跳变检查

```

-> 速度20点窗口、方差、自适应滤波、死区
-> GPS_DATA

超过 3 秒无可解析数据会标记丢失；每 10 秒发送热启动和恢复配置。恢复配置将更新周期设置为 5 Hz，与启动时 10 Hz 不一致。

10.2 Web

ESP32 创建 ESP32-IMU 热点，密码硬编码为 12345678，地址通常为 192.168.4.1。

路由	作用
/	从 SPIFFS 流式返回 index.html
/data	通用速度、GPS、G 力、性能 JSON
/moto	摩托倾角、事件和峰值 JSON
/audio	录音状态 JSON
/audio/control?action=...	开始、停止、暂停、恢复、删除、自动模式、列表
/audio/download?file=...	下载 WAV

网页每 150 ms 请求 /data，每 100 ms 请求 /moto。这是轮询架构，简单但请求频繁，且 /moto 请求本身会再次调用 updateMotorcycleData()，使算法更新时间受浏览器请求节奏影响。

11 状态机

11.1 音频录制

```
IDLE/STOPPED -- speed >= 5 --> ACTIVE
ACTIVE -- speed < 2 for 5s --> STOPPED
ACTIVE -- pause -----> PAUSED
PAUSED -- resume -----> ACTIVE
ACTIVE -- 300s -----> STOPPED
```

RECORDING_WAITING 被定义但当前逻辑基本未使用。

11.2 性能测试

- 加速：速度超过 5 km/h 自动开始，穿越 60/100/200 km/h 时记录时间。
- 制动：速度达到约 95 km/h 后进入条件，降到 5 km/h 结束。
- 圈速字段和函数声明存在，但完整实现无法从当前工作区确定。

12 第四部分：核心模块解析

13 main.cpp：系统装配与输出层

职责：

- 创建两条 TwoWire 总线、OLED、WebServer。
- 固定初始化顺序和主循环频率。
- 把全局数据编码为 OLED 页面和 JSON。
- 管理 BOOT 按键及串口模式。

关键设计是多速率协作循环。最大问题是文件已超过千行，HTTP、OLED、调度和业务拼在一起，修改一个输出字段容易影响实时路径。

14 mpu6050：高振动车辆 IMU 前端

核心结构 MPU6050_DATA 同时保存原始和滤波后的加速度、角速度、Roll/Pitch。

处理链：

Adafruit getEvent()

-> 10样本滑动平均

-> $\alpha=0.1$ 一阶IIR

-> 加速度倾角

量程为 $\pm 8\text{ g}$ 、 $\pm 500\text{ deg/s}$ ，DLPF 为 94 Hz。选择 94 Hz 是为了保留快速车辆动作，再依赖软件滤波压制振动。

注意：

- CalculateAttitude() 的基础 Roll/Pitch 仅使用加速度计。
- Yaw 固定为 0。
- 陀螺数据在 Adafruit API 中是 rad/s，但摩托互补滤波直接按 deg/s 积分的迹象很强；若未转换，倾角动态项会缩小约 57.3 倍。需要实机与库单位确认后修正。
- 初始滤波状态为 0，会有启动爬升过程。

15 gps_module：GPS、速度与性能计时

核心结构：

- GPS_DATA：位置、时间、速度、信号质量、融合状态。
- PERFORMANCE_DATA：加速、制动、圈速和会话数据。

主要策略:

- 仅输出 RMC/GGA, 降低 9600 bps 下 NMEA 堵塞风险。
- 开启 SBAS/WAAS。
- 对位置做经纬度范围和 1 km 单帧跳变检查。
- 对速度做 20 点窗口、标准差、卫星/HDOP 动态阈值、分段 IIR 和最终死区。
- 静止时收集 200 个样本估计 IMU 前向偏置。
- IMU 积分速度每次 GPS 新帧向 GPS 拉回 10%。

设计意图正确, 但实现中 `gps_data.speed_ms` 很少被明确同步为当前 GPS 速度, 而校准与静止归零依赖它; 应统一只使用 `speed_mps` 或明确维护该字段。

16 performance_analyzer: 派生指标与摩托语义

FUSED_PERFORMANCE_DATA 保存当前速度/G 力、峰值、行为状态和质量分数。

主要功能:

- 组合 IMU 加速度与 GPS 速度差分。
- 检测加速、制动、转弯。
- 用车辆质量、滚阻、风阻估算瞬时功率。
- 用互补滤波估计摩托倾角和俯仰。
- 检测弯道、翘头、翘尾, 估算弯道半径。

这里把“信号处理”和“车辆业务语义”放在同一模块, 原型阶段方便, 但未来应拆成:

Sensor Fusion

Vehicle Frame Transform

Event Detector

Session Statistics

硬编码偏置 -0.47/+0.5/-0.48 和假定坐标 X=垂直、Y=侧向、Z=前向 是整个分析可信度的基础。换安装方向或换板后必须重新标定。

17 audio_recorder: 低码率事件录音

配置为单声道、8 kHz、8 bit PCM, 每秒约 8 KB, 5 分钟约 2.4 MB, 不含文件系统开销。

WAV 头在开始时写占位值, 停止时回写文件大小。这种方式简单, 但异常断电会留下头部未完成的文件。

最大架构问题是 ISR 只有一个样本槽, 不是 DMA/环形缓冲。录音数据完整性无法保证。

18 sd_logger: 行驶 CSV

SD 使用 HSPI:

MISO GPIO25

MOSI GPIO13

SCK GPIO14

CS GPIO15

选择 GPIO25 替代 GPIO12, 是为了规避 ESP32 启动绑带引脚风险。日志以 10 Hz 写 CSV, 每 100 点 flush 一次, 在数据安全和写入开销之间折中。

当前 setup() 没有调用 initSDCard(), 因此 checkAutoLogging() 会因 initialized=false 直接返回。

19 data/index.html: 无 App Web 仪表盘

单文件前端包含:

- 实时速度/G 力曲线。
- 性能、GPS、摩托状态。
- 浏览器端会话记录与 CSV/JSON 导出。
- 录音控制逻辑。

优点是部署简单。缺点是 HTML 已很大、编码损坏明显、无构建/校验、API 字段靠手写约定。浏览器记录只存在内存, 刷新即丢失。

20 cad/pcb1_enclosure.scad: 参数化产品外壳

外壳从 Gerber 轮廓推导出 78.867 x 70.866 mm、圆角 5.08 mm 的 PCB:

- 底壳用周边支撑环和卡扣固定无安装孔 PCB。
- 上盖使用局部器件避让腔。
- 四角外置螺钉耳避免占用板内空间。
- 提供通风槽、边缘开口、装配/爆炸/导出模式。

器件高度和接口位置是从丝印/飞针数据推断的, 不是装配实测。打印前必须实测校核。

21 第五部分：技术知识提取

22 双 I2C

22.1 为什么需要它

MPU6050 与 OLED 本可共享地址不同的总线，但 OLED 整帧刷新会占用 I2C 时间。分成两条控制器可以隔离显示流量与传感器采样，也方便独立排错。

22.2 项目中的实际应用

- I2C0: GPIO21/22, MPU6050。
- I2C1: GPIO18/19, SSD1306。
- 两者均配置 400 kHz。

22.3 常见 Bug 与调试

- 文档接线漂移是当前首要风险。
- 地址应扫描确认：MPU 0x68/0x69, OLED 0x3C/0x3D。
- 线长、上拉、电源噪声会导致间歇 NACK。
- 用现有 `scanI2CDevices()` 先区分接线问题与驱动问题。

替代方案是共享一条 I2C，节省引脚；对本项目而言隔离总线更适合实时采样。

23 UART 与 NMEA

GT-U7 通过 UART2 输入 NMEA。项目只保留 RMC 与 GGA，因为 10 Hz、9600 bps 时全量语句容易占满串口带宽。

常见坑：

- 模块不一定兼容所有 PMTK 命令。
- 发送 10 Hz 配置不代表模块确实进入 10 Hz。
- 掉线恢复又设置 5 Hz，导致运行状态不一致。
- 应统计每秒完整 RMC/GGA 帧数，而不是只相信配置命令。

24 采样与数字滤波

项目使用了三类低成本滤波：

- 硬件 DLPF：在传感器内部先限制带宽。
- 滑动平均：压制随机噪声，但引入窗口延迟。
- 一阶 IIR：O(1) 状态，适合 MCU，但有相位延迟。

常见误区是把“200 Hz 调用周期”当成严格采样率。当前系统是协作循环，实际采样间隔会抖动，且代码给融合器的 Δt 固定为 5 ms，没有用真实时间差。

调试时应同时记录：

- 实际 `micros()` 间隔分布。
- 原始值、滑动平均值、IIR 值。
- 静止 RMS、阶跃响应和相位延迟。

25 GPS 静止漂移

GPS 低速时噪声可能大于真实速度。项目不是简单小于阈值清零，而是综合窗口方差、连续稳定次数、卫星数和 HDOP。

这能改善显示，但阈值滤波也会吃掉真实低速起步，使 0-100 起点变晚。性能计时最好保留原始 Doppler 速度和滤波速度两路，分别服务计时与 UI。

26 GPS-IMU 融合

当前思路：

IMU: 高频预测，但漂移

GPS: 低频校正，但延迟和低速噪声

项目采用简化互补校正而非 EKF，优点是参数直观、计算低。缺点是没有显式状态协方差、时间戳对齐、姿态重力补偿和传感器延迟模型。

真正升级时，优先实现一维状态 [速度, 加速度偏置] 的 Kalman filter，而不是直接上完整 9/15 状态导航 EKF。

27 ADC、Timer 与音频

定时器中断可以建立稳定采样节拍，但 ISR 应只搬运数据，不能依赖单槽共享变量承载连续流。

更合适的方案：

- ESP32 I2S ADC/DMA。
- ISR 写入 lock-free ring buffer。
- 双缓冲，由任务批量写文件。

此外 8-bit unsigned PCM 的直流中心假设是 128；模拟麦克风偏置和 ADC 非线性应实测。

28 SPI、SD 与文件系统

SD 写入延迟具有长尾，`flush()` 可能阻塞几十毫秒。若和 200 Hz 采样同循环，必须缓冲解耦。

SPIFFS 适合少量静态资源和小文件，不适合高频持续音频写入。LittleFS 或 SD 更适合长期录音；异常断电还需要临时文件和恢复策略。

29 Wi-Fi 与 HTTP

SoftAP 让手机直接连接，现场无需路由器，这是很实用的产品决策。

当前轮询方案的常见问题：

- 多客户端会放大 CPU 和 JSON 序列化开销。
- 同步 WebServer 的流式下载可能阻塞采样。
- `sprintf` 固定缓冲区依赖字段长度，需用 `snprintf`。
- 密码硬编码，适合原型但不适合发布。

替代方案：Server-Sent Events 或 WebSocket 推送，并将静态资源缓存与实时数据通道分离。

30 第六部分：设计决策分析

31 双 I2C 而不是共享总线

背景：显示更新与高频 IMU 读取争用总线。

优点：故障隔离、时序可控、扫描清晰。缺点：多占两根 GPIO，接线复杂。当前 ESP32 引脚尚可，选择合理。

32 协作式调度而不是 FreeRTOS 任务

背景：Arduino 原型需要快速迭代。

优点：控制流直观、调试容易。缺点：所有模块共享阻塞预算，无法保证 200 Hz IMU 和 8 kHz 音频。

当前方案适合功能验证；当音频、SD 和网络同时开启时，应迁移为任务和队列：

Core/Task Sensor -> Queue -> Fusion

Core/Task IO -> SD/SPIFFS/Web

33 多级滤波而不是单一滤波器

车辆振动覆盖宽频段，单一强低通会造成明显延迟。硬件 DLPF、移动平均和 IIR 分层容易调参，但当前级联实际带宽没有通过测试数据验证。

替代方案包括双二阶 Butterworth、陷波器、Savitzky-Golay 或频域分析后定制滤波。应先采集原始数据做 PSD，再决定，而不是继续堆滤波。

34 互补滤波而不是 EKF

当前传感器精度有限，且原型更需要可解释性。互补滤波成本低、故障容易定位。

缺点是重力投影、安装误差和时间同步需要手工处理。等硬件、坐标系和时间戳稳定后，再引入一维 KF 或姿态 EKF。

35 阈值状态机

自动录音、SD 日志、加速/制动、弯道检测都采用阈值 + 延时/滞回。

优点是行为可解释；缺点是阈值分散在多个文件，且录音与 SD 的起停参数不一致：

- 录音停止延时 5 秒。
- SD 停止延时 30 秒。

应集中成 VehicleSessionConfig，并用一次会话状态驱动所有记录器。

36 浏览器端导出

浏览器记录减轻 ESP32 存储压力，开发方便。但手机锁屏、刷新、断连都会丢数据，不能替代设备端可靠日志。它更适合作为“即时观察与临时导出”。

37 第七部分：调试与踩坑记录

38 调试记录 1：文档与固件接线冲突

问题：按旧 README 接线可能找不到设备。

原因：项目从共享 I2C 演进到双 I2C，文档未统一。

定位：以 main.cpp 宏和启动扫描输出为准。

解决：维护唯一 HARDWARE_PINOUT.md，其他文档只链接，不复制引脚表。

经验：硬件配置必须单一事实源。

39 调试记录 2：音频丢采样

问题：WAV 时长、音质或样本数异常。

原因：8 kHz ISR 覆盖单个 lastSample；主循环无法保证每 125 us 消费。

定位：比较墙钟时长与 samples_recorded / 8000，统计覆盖次数。

解决：I2S DMA 或环形缓冲，文件写入放到独立任务。

40 调试记录 3：SD 功能看似存在但不工作

问题：速度超过阈值仍不生成 CSV。

原因：initSDCard() 在 setup() 被注释，sd_status.initialized 始终为 false。

定位：查看启动日志是否打印 SD 容量。

解决：恢复初始化前先解决原注释所说的 I2C/引脚冲突，并增加 Web/串口 SD 状态。

41 调试记录 4：倾角动态响应可能错误

问题：陀螺贡献过弱，倾角主要跟随加速度计。

原因：Adafruit 陀螺输出通常为 rad/s，而摩托代码按 deg/s 注释直接积分。

定位：固定角速度转动，比较原始 gyro、积分角与实际角度。

解决：在驱动边界统一单位，字段命名加入 _rad_s 或 _deg_s。

42 调试记录 5: GPS 模式不一致

问题: 掉线恢复后刷新率下降, 性能测试响应变化。

原因: 启动发送 100 ms 更新命令, 恢复发送 200 ms 更新命令。

解决: 抽取 `configureGPS()`, 启动和恢复调用同一配置; 读取实际帧率确认结果。

43 调试记录 6: 启动顺序与 SPIFFS

问题: 音频模块初始化时容量可能为 0 或状态不可靠。

原因: 音频初始化早于 `SPIFFS.begin(true)`。

解决: 先初始化存储, 再初始化依赖存储的音频和 Web。

44 调试记录 7: 融合字段与 API 不一致

问题: 内部已经运行速度融合, 但网页 `imuSpeed` 永远为 0, `speed` 仍来自 GPS。

原因: 算法、状态结构和 JSON 输出没有同时更新。

解决: 定义明确字段: `gps_speed`、`imu_predicted_speed`、`fused_speed`, 并建立 API schema 测试。

45 调试记录 8: HTTP 文件下载路径

问题: 下载接口可能访问任意 SPIFFS 已知路径, 而不仅是录音。

原因: 只补 / 和检查文件存在, 没有限定 `/ride_*.wav`。

解决: 白名单文件名格式, 拒绝 ..、非 WAV 和非录音前缀。

46 调试记录 9: 字符串与缓冲区

问题: JSON/文本字段增长后可能溢出固定缓冲区。

原因: 使用 `sprintf` 而非长度受限接口。

解决: 改为 `snprintf` 并检查返回值; 长期使用 `ArduinoJson` 流式序列化。

47 调试记录 10: 编码损坏

问题: 中文日志、网页标签和文档显示乱码, 甚至可能破坏 HTML 标签或 C++ 字符串。

原因: 文件编码链不一致。

解决: 全仓统一 UTF-8, 无 BOM; 添加 .editorconfig, 在 CI 中做 UTF-8 解码和 HTML/C++ 基础检查。

48 第八部分: 工程实践总结

48.1 做得好的地方

- 模块按传感器和业务域拆分, 原型意图清楚。
- 使用 `millis()` 而不是在主路径大量固定延时。
- 对 GPS 弱信号、静止漂移和跳点有工程化防护。
- 同时保留原始值、滤波值和峰值, 便于调试。
- 通过 SoftAP + 单页 Web 降低现场使用门槛。
- SD 选择 HSPI 和非 GPIO12 MISO, 体现了对 ESP32 启动绑带的考虑。
- OpenSCAD 参数化外壳把软件项目向可装配设备推进了一步。

48.2 可以改进的地方

48.2.1 代码组织

`main.cpp` 应拆成:

```
app_scheduler
web_api
oled_view
hardware_config
```

全局 `extern` 状态应逐步替换为只读快照或消息结构, 至少明确谁写、谁读、更新频率和单位。

48.2.2 配置管理

引脚、采样率、阈值、车辆参数和 Wi-Fi 凭据分散在各文件。建议集中:

```
board_config.h
sensor_config.h
vehicle_config.h
```

并把所有物理量写入单位后缀。

48.2.3 日志

串口日志丰富，但缺乏级别和节流统一。应使用 LOGE/W/I/D，避免高频 Serial.printf 破坏实时性。

48.2.4 错误处理

很多初始化失败后继续运行，这是原型的容错思路，但后续函数仍可能假定硬件有效。应维护模块健康状态，并在 UI 显示 degraded mode。

48.2.5 测试策略

当前 test/ 只有模板说明，没有项目测试。最值得先补的是纯算法主机测试：

- Haversine 距离。
- GPS 窗口与静止判定。
- 阈值穿越计时。
- WAV 头尺寸。
- CSV 格式。
- 互补滤波单位与步进响应。
- 状态机起停滞回。

硬件在环测试应回放已保存的 NMEA 与 IMU 数据，而不是每次都上车测试。

48.2.6 构建与 CI

当前配置锁定库版本是好事。建议 CI 至少执行：

```
pio run
pio test -e native
HTML/JSON schema sanity check
UTF-8 validation
```

本次分析环境无法执行构建，因为终端没有可用的 pio 命令；因此当前源码是否能完整编译无法从本次静态分析确定。

49 第九部分：项目复盘

49.1 最大的亮点

不是简单堆传感器，而是围绕“低成本硬件在真实车辆环境中不可靠”做了多层处理：信号质量、静止抑制、漂移校准、融合、状态机和多终端显示。

49.2 最大的难点

多时间尺度和多误差源同时存在：

- 音频 8 kHz。
- IMU 200 Hz。
- 显示/融合 20 Hz。
- GPS/日志 10 Hz。
- Web 约 6.7/10 Hz。

而它们目前共享一个可能阻塞的主循环。真正的系统瓶颈是调度与数据一致性，不再只是算法。

49.3 学到的知识

- 传感器数据“能读”与“能用于决策”之间隔着校准、坐标系、滤波、质量评估和时间同步。
- 低成本 GPS 的极限不能只靠软件宣传突破，升级硬件有时比继续堆阈值更合理。
- 原型文档很容易落后于代码，尤其是引脚和单位。
- 记录系统必须考虑掉电、断连和阻塞长尾。
- 机械结构、接线和软件配置应被视为同一系统架构。

49.4 如果重新设计

1. 先定义统一硬件配置、坐标系和单位。
2. 使用 I2S DMA 采音，传感器和存储通过队列解耦。
3. 建立单一 VehicleSession 状态机，统一音频、CSV 和峰值会话。
4. 所有数据携带 micros() 时间戳，融合使用真实 dt。
5. 保留 raw/filtered/fused 三层数据并定义稳定 API schema。
6. 先做数据回放工具和 native 单元测试，再调算法。
7. 用实测频谱和标定数据选择滤波器。
8. 把网页通信改成 SSE/WebSocket。

49.5 下一步扩展

优先级建议：

1. 修复编码、引脚文档和单位问题。
2. 恢复可验证的 SD 日志，获得真实数据集。
3. 修复音频采样架构。
4. 增加时间戳和离线回放测试。
5. 一维速度/偏置 Kalman filter。
6. 升级 NEO-M9N 或更高频 GNSS；是否上 RTK 取决于实际计时精度目标。
7. 增加 OTA、配置页面和会话下载。

无法从当前工作区确定：现有 PCB 是否已经覆盖麦克风与 SD 模块、外壳是否完成实物试装、车辆参数是否经过标定、GPS 优化报告中的量化提升是否来自可复现实测。

50 第十部分：个人知识地图

51 Embedded Knowledge Map

嵌入式系统

- ├─ ESP32
 - | └─ Arduino Framework
 - | └─ GPIO与启动绑带
 - | └─ 双I2C控制器
 - | └─ UART2
 - | └─ HSPI
 - | └─ ADC1与硬件Timer
 - | └─ SPIFFS
 - | └─ Wi-Fi SoftAP / HTTP
- ├─ 传感器与信号处理
 - | └─ MPU6050量程与DLPF
 - | └─ 滑动平均
 - | └─ 一阶IIR
 - | └─ 互补滤波
 - | └─ 零偏与安装校准
 - | └─ 坐标系映射
 - | └─ 重力补偿
 - | └─ 采样率、抖动与真实dt
- ├─ GNSS
 - | └─ NMEA RMC/GGA
 - | └─ Doppler速度
 - | └─ 卫星数与HDOP
 - | └─ 静止漂移抑制
 - | └─ 跳点过滤

- | |— SBAS
- | |— 多频率数据融合
- | |— GT-U7到高性能GNSS升级
- |
- |— 车辆与机器人数据
 - | |— 纵向/横向/垂直G
 - | |— 加速与制动计时
 - | |— 倾角与俯仰
 - | |— 弯道检测与半径估计
 - | |— 翘头/翘尾事件
 - | |— 峰值与会话统计
 - | |— 简化功率估算
- |
- |— 数据记录
 - | |— WAV/PCM
 - | |— CSV
 - | |— JSON API
 - | |— 环形缓冲与DMA
 - | |— SD写入长尾
 - | |— 掉电一致性
 - | |— 浏览器端临时记录
- |
- |— 软件架构
 - | |— 协作式调度
 - | |— 多速率任务
 - | |— 状态机与滞回
 - | |— 全局状态耦合
 - | |— 消息队列/生产者消费者
 - | |— 模块健康状态
 - | |— API schema
- |
- |— 工程化
 - | |— PlatformIO依赖锁定
 - | |— Native算法测试
 - | |— 硬件在环与数据回放
 - | |— 日志分级
 - | |— UTF-8与文档单一事实源
 - | |— CI构建与静态检查
- |
- |— 产品化
 - | |— Gerber与PCB制造资料
 - | |— 飞针测试数据
 - | |— OpenSCAD参数化外壳
 - | |— 装配公差与器件避让
 - | |— Web免安装交互
 - | |— 硬件成本与精度边界

51.1 长期维护入口

半年后重新开始时，建议按这个顺序：

1. 先读本文的“阅读前事实”和“调试记录”。
2. 核对实际硬件接线与 `main.cpp` 引脚。
3. 执行 PlatformIO 构建并保存构建环境版本。
4. 串口确认 I2C 扫描、GPS 帧率、SPIFFS 和 SD 状态。
5. 先记录一份 raw 数据，再修改滤波或融合参数。
6. 所有新的实测结论更新到本文，同时删除或标记过时文档中的冲突说法。